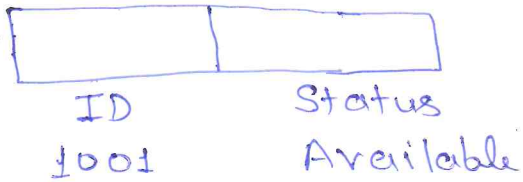
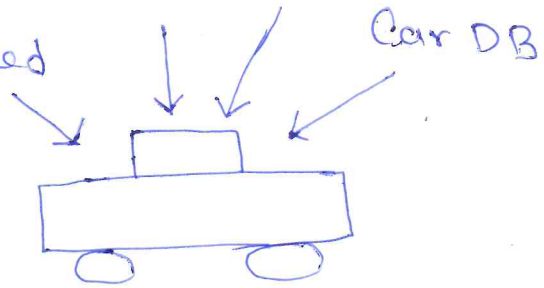
Critical Section

Code Segment, where ~~should~~ shared resources are being accessed and modified.




```
{
Read Car Row with id : 1001
if Status is Available :
    Update it to Booked
}
```



When a multiple request try to access this critical section, Data Inconsistency can happen.

Its solution is usage of TRANSACTION

Concept && Coding

TRANSACTION

It's helps to achieve ACID property.

A (Atomicity) :→ Ensures All operations within a transaction are completed successfully. If any operation fails, entire transaction will get rollback.

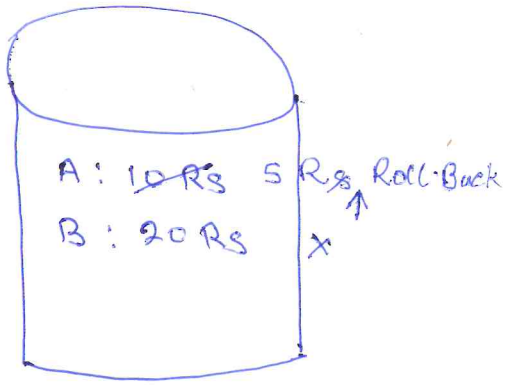
{

- Debit A 5 Rs

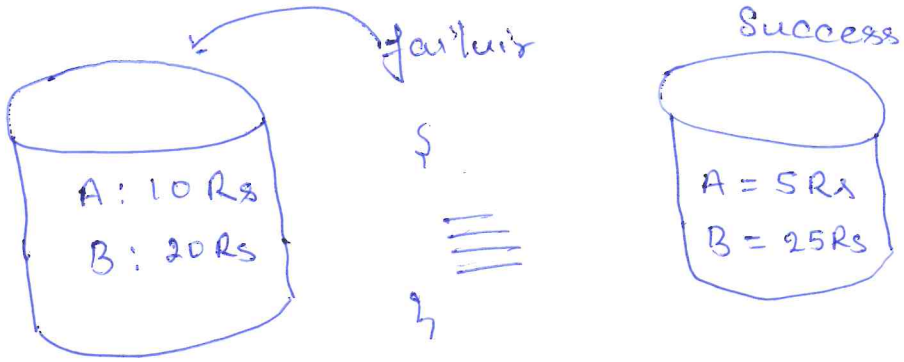
  Credit B 5 Rs

}

Txn 1



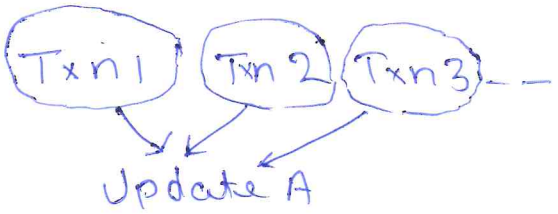
C (Consistency) :→ Ensures that DB state before and after the transactions should be Consistent Only.



Concept && Coding

I (Isolation): Ensure that, even if multiple transactions are running in parallel, they do not interface with each other.

Txn 1	Txn 2	Txn 3
{	{	{
Debit A: 5Rs	Debit A : 2Rs	Credit A : x
Credit B: 5R	Credit B : 2Rs	Dedit B : y
}	}	}
COMMIT		



D (Durability): Ensures that committed transaction will never lost despite system failure or crash.

```
BEGIN_TRANSACTION:
operation { - Debit from A
            - Credit to B
            if all success:
                COMMIT;
            Else
                ROLLBACK;
END_TRANSACTION
```

Concept && Coding

In Springboot, we can use @Transactional annotation.

AND for that

1:- we need to add below Dependency in pom.xml.
(based on DB we are using, suppose we are using RELATIONAL DB)

* Spring boot Data JPA (Java Persistence API): helps to interact with Relational database without

spring-boot-starter-data-JPA

* Database driver dependency is also required

2:- Active, Transaction Management by using @EnableTransactionManagement in main class.
(Sprig boot generally Auto Configuration it, so we don't need to specially add it)

@Transactional

At class level
- Transaction applied to all Public method

At method level
- Transaction applied to particular method only.

Concept && Coding

Transaction Managemat in Spring boot uses ADP

Pontcut — Advise — Before, After, Around

→ 1:- Uses Point cut expression to search for method, which has @Transactional annotation like.

@within(org.springframework.transaction.annotati.Transactional)

2:- Once Point cut expression matches, run and "Around" type Advice. → After, Before, Around → executes before and after join point

Advice is:

invokeWithinTransaction method present in TransactionalInterceptor class

Concept && Coding